

Section A: Artificial Intelligence

Question 1

- a) Define Artificial Intelligence (AI) and explain its main types with suitable examples.
 - b) What tasks can AI perform in today's world?
 - c) When performing an action, how does a reflex agent differ from a planning agent? Explain with an example.
-

a) Define Artificial Intelligence (AI) and explain its main types with suitable examples.

Definition of Artificial Intelligence (AI)

Artificial Intelligence (AI) is a branch of computer science concerned with the design and development of computer systems that can perform tasks which normally require human intelligence. These tasks include learning from experience, reasoning, problem-solving, perception, decision-making, and understanding natural language. In simple words, AI enables machines to think, learn, and act intelligently like humans.

Main Types of Artificial Intelligence

Artificial Intelligence can be broadly classified into the following types based on **capability**:

1. Narrow AI (Weak AI)

Narrow AI is designed to perform **one specific task** or a limited range of tasks. It operates under predefined rules and does not possess real understanding, consciousness, or self-awareness. Narrow AI works only within its programmed domain and cannot perform tasks beyond that.

Most AI systems available today fall into this category.

Characteristics:

- Task-specific
- Cannot perform outside its defined domain
- No real understanding or consciousness
- Most common form of AI today

Examples:

- Google Search Engine
- Face recognition systems
- Chatbots and virtual assistants
- Email spam filters
- Recommendation systems (Netflix, YouTube)
- Siri, Google Assistant, Face Recognition, ChatGPT.

2. General AI (Strong AI)

General AI refers to machines that can perform human-like intelligence. Such systems can understand, learn, reason, and apply knowledge across multiple domains, similar to human beings. A General AI system would be able to perform **any intellectual task** that a human can do.

Characteristics:

- Can reason, plan, and learn independently
- Can solve unfamiliar problems
- Capable of human-level thinking and understanding

Examples:

- Does not currently exist
- Remains theoretical
- Subject of advanced research in AI
- No real-world example yet; movie examples include JARVIS (Iron Man).

3. Super AI

Super AI is a **hypothetical** form of artificial intelligence that becomes more intelligent than humans in every aspect, including creativity, emotions, problem-solving, and decision-making. It represents a future stage where AI becomes more intelligent than humans in all fields.

Characteristics:

- More intelligent than humans
- Self-aware
- Capable of independent innovation and decision-making

Examples:

- Exists only in science fiction
- Discussed in future research and theoretical studies
- No real examples; fictional ones include Ultron and Terminator's Skynet.

b) What tasks can AI perform in today's world?

In the modern world, AI is widely used in almost every sector. Some important tasks performed by AI are:

1. Data Processing and Analysis

- Analyzing large volumes of data
- Identifying patterns and trends

- Used in business analytics and scientific research

2. Natural Language Processing (NLP)

- Speech recognition
- Language translation
- Chatbots and virtual assistants
- Text summarization

3. Computer Vision

- Face and object recognition
- Medical image analysis (X-rays, MRI scans)
- Surveillance systems

4. Automation and Robotics

- Industrial robots in manufacturing
- Automated warehouses
- Assembly line operations

5. Healthcare

- Disease diagnosis
- Predicting patient outcomes
- Drug discovery
- Medical decision support systems

6. Transportation

- Self-driving cars
- Traffic management
- Route optimization

7. Finance

- Fraud detection
- Credit scoring
- Algorithmic trading

8. Education and Entertainment

- Personalized learning platforms
- Intelligent tutoring systems
- Game AI

- Content recommendation

9. Security and Defense

- Threat detection
- Cybersecurity systems
- Surveillance analysis

c) When performing an action, how does a reflex agent differ from a planning agent? Explain with an example.

Reflex Agent

A Reflex Agent is the simplest type of intelligent agent in Artificial Intelligence. It selects actions solely based on the current percept using predefined condition–action rules, without considering past experiences, internal states, or future consequences. Reflex agents do not learn from experience and respond immediately to environmental stimuli, making them suitable only for fully observable and static environments.

Characteristics:

- Reacts instantly
- No memory or planning
- Simple and fast
- Limited intelligence

Example:

A thermostat:

- If temperature is low → heater ON
- If temperature is high → heater OFF

It does not think about future temperature changes.

Planning Agent

A Planning Agent is an AI agent that thinks before acting. Instead of reacting immediately, it analyzes the environment, predicts future outcomes, and creates a sequence of actions (a plan) to achieve a goal in the most efficient way.

Characteristics:

- Goal-oriented
- Uses reasoning and planning
- Considers future states
- More intelligent but slower

Example:

A self-driving car:

- Plans the shortest route
- Avoids traffic
- Follows traffic rules
- Predicts future road conditions

Difference Between Reflex Agent and Planning Agent

Feature	Reflex Agent	Planning Agent
Decision Basis	Current perception only	Future goals and outcomes
Memory	No	Yes
Intelligence	Low	High
Flexibility	Very limited	Highly flexible
Example	Thermostat	Self-driving car

Question 2

- a) What is a **search problem in AI**? Explain the **main components of a search problem**.
- b) Explain the difference between a **state space graph** and a **search tree** in AI.
- c) Consider a **Tic-Tac-Toe game** with the following information:

- Number of agent positions: 120
- Number of food dots: 30
- Number of ghost positions: 12
- Agent facing directions: North, South, East, West

Now calculate:

1. The **number of world states**
2. The **number of states for pathing**
3. The **number of states for eat-all-dots**

a) What is a search problem in AI? Explain the main components.

A search problem in AI is a problem where an agent tries to find a sequence of actions that leads from an initial state to a goal state.

The agent searches through different possible states to find a solution.

1. Initial State

The **initial state** represents the starting point of the problem. It is the state from which the agent begins its search for a solution.

2. State Space

The **state space** is the complete set of all possible states that the agent can reach from the initial state by applying actions. It defines every configuration the problem can take.

3. Actions (Successor Function)

Actions, also called the successor function, describe what moves or operations are possible from a given state and show which new state the system reaches after performing an action.

4. Goal Test

The **goal test** checks whether a given state satisfies the goal condition. It is used to determine if the search has successfully reached a solution.

5. Path Cost (Optional)

The **path cost** measures the cost of moving from one state to another. It is often used to evaluate and compare different solution paths, especially in optimal search problems.

✦ Example:

In a maze, the initial state is the start position, the goal is the exit, actions are moves (up, down, left, right), and the state space is all reachable positions.

b) Difference between a state space graph and a search tree

Aspect	State Space Graph	Search Tree
Definition	Complete model of the problem	Exploration structure used by an algorithm
Node meaning	Unique problem state	State + specific path
Duplicate states	Not allowed	Allowed
Creation	Defined before search	Built during search
Algorithm dependence	Independent	Dependent
Size	Finite or infinite	Can grow much larger
Cycles	Can exist	Can cause infinite expansion
Purpose	Problem representation	Problem-solving process

A State Space Graph is a graphical representation of all possible states of a problem and the transitions (actions) between those states.

c) Given information

- Number of agent positions = **120**
- Number of food dots = **30**
- Number of ghost positions = **12**
- Agent facing directions = **4** (N, S, E, W)

1 Number of world states

A **world state** includes:

- Agent position
- Food configuration
- Ghost positions
- Agent facing direction

Formula:

$$\begin{aligned}\text{World States} &= (\text{Agent positions}) \times (\text{Food states}) \times (\text{Ghost positions}) \times (\text{Directions}) \\ &= 120 \times 2^{30} \times 12 \times 4\end{aligned}$$

$$120 \times 12 \times 4 \times 2^{30}$$

📌 Food dots can be **present or eaten**, so total food configurations = 2^{30} .

2 Number of states for pathing

Pathing problem:

- Agent only needs to move from start to goal
- **Food , Facing direction and ghosts are ignored**

State includes:

- Agent position

Formula:

$$\begin{aligned}\text{Pathing States} &= (\text{Agent positions}) \\ &= 120\end{aligned}$$

3 Number of states for eat-all-dots

For eat-all-dots, we only care about:

- Agent position: 120 options
- Food configuration: 2^{30} options

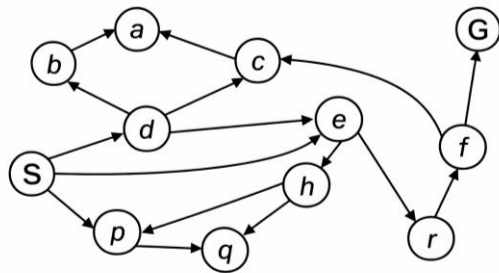
We ignore agent direction and ghost positions.

Total eat-all-dots states = 120×2^{30}

$= 120 \times 1,073,741,824$

$= 128,849,018,880$ ($\approx 1.29 \times 10^{11}$ states)

Question 3



a) Apply **Depth-First Search (DFS)** to the given graph and show the traversal order clearly.

b) Answer the following questions related to **Breadth-First Search (BFS)**:

1. How much **space** does the **fringe** require?
2. Is BFS **complete**?
3. Is BFS **optimal**?

a) Apply Depth-First Search (DFS)

1 Correct Graph (as YOU defined)

Directed edges:

- $S \rightarrow d$
- $S \rightarrow e$
- $S \rightarrow p$
- $d \rightarrow b$
- $d \rightarrow c$
- $d \rightarrow e$
- $b \rightarrow a$
- $c \rightarrow a$
- $e \rightarrow h$
- $e \rightarrow r$
- $h \rightarrow p$
- $h \rightarrow q$

- $p \rightarrow q$
- $r \rightarrow f$
- $f \rightarrow c$
- $f \rightarrow G$

✓ This is now our **only source of truth**.

2 DFS Assumptions (important for exams)

- **Start node: S**
- Use **stack-based DFS**
- When multiple neighbors exist → **visit in the order listed**
- **Do not revisit** visited nodes

3 Step-by-Step DFS Process with Stack

Step 1: Start at S

Stack: [S]

Visited: {S}

Action: Visit S

Step 2: From S, push d

Stack: [S, d]

Visited: {S, d}

Action: Visit d

Step 3: From d, push b

Stack: [S, d, b]

Visited: {S, d, b}

Action: Visit b

Step 4: From b, push a

Stack: [S, d, b, a]

Visited: {S, d, b, a}

Action: Visit a

Step 5: a has no outgoing edges → Backtrack

Stack: [S, d, b]

Step 6: b has no unvisited neighbors → Backtrack

Stack: [S, d]

Step 7: From d, push c

Stack: [S, d, c]

Visited: {S, d, b, a, c}

Action: Visit c

Step 8: c → a (already visited) → Backtrack

Stack: [S, d]

Step 9: From d, push e

Stack: [S, d, e]

Visited: {S, d, b, a, c, e}

Action: Visit e

Step 10: From e, push h

Stack: [S, d, e, h]

Visited: {S, d, b, a, c, e, h}

Action: Visit h

Step 11: From h, push p

Stack: [S, d, e, h, p]

Visited: {S, d, b, a, c, e, h, p}

Action: Visit p

Step 12: From p, push q

Stack: [S, d, e, h, p, q]

Visited: {S, d, b, a, c, e, h, p, q}

Action: Visit q

Step 13: q has no neighbors → Backtrack

Stack: [S, d, e, h, p]

Step 14: p has no more neighbors → Backtrack

Stack: [S, d, e, h]

Step 15: h → q already visited → Backtrack

Stack: [S, d, e]

Step 16: From e, push r

Stack: [S, d, e, r]

Visited: {S, d, b, a, c, e, h, p, q, r}

Action: Visit r

Step 17: From r, push f

Stack: [S, d, e, r, f]

Visited: {S, d, b, a, c, e, h, p, q, r, f}

Action: Visit f

Step 18: From f, push c (already visited → skip)

Step 19: From f, push G

Stack: [S, d, e, r, f, G]

Visited: {S, d, b, a, c, e, h, p, q, r, f, G}

Action: Visit G

Step 20: G has no neighbors → Backtrack fully

Stack: []

✅ DFS COMPLETE

🔑 Final DFS Traversal Order (Correct)

$S \rightarrow d \rightarrow b \rightarrow a \rightarrow c \rightarrow e \rightarrow h \rightarrow p \rightarrow q \rightarrow r \rightarrow f \rightarrow G$

BFS for Practice

1 Graph Used (same as DFS)

Directed edges (given):

- $S \rightarrow d, e, p$
- $d \rightarrow b, c, e$
- $b \rightarrow a$
- $c \rightarrow a$
- $e \rightarrow h, r$
- $h \rightarrow p, q$
- $p \rightarrow q$
- $r \rightarrow f$
- $f \rightarrow c, G$

2 BFS Assumptions (write in exam)

- **Start node:** S
- **Data structure:** Queue (FIFO)
- Visit neighbors **in the given order**
- Do **not revisit** visited nodes

3 Step-by-Step BFS Process with Queue

Step 1: Start at S

Queue: [S]

Visited: {S}

Action: Visit S

Step 2: Dequeue S, enqueue its neighbors (d, e, p)

Queue: [d, e, p]

Visited: {S, d, e, p}

Step 3: Dequeue d, enqueue its unvisited neighbors (b, c)

Queue: [e, p, b, c]

Visited: {S, d, e, p, b, c}

Step 4: Dequeue e, enqueue its unvisited neighbors (h, r)

Queue: [p, b, c, h, r]

Visited: {S, d, e, p, b, c, h, r}

Step 5: Dequeue p, enqueue its unvisited neighbor (q)

Queue: [b, c, h, r, q]

Visited: {S, d, e, p, b, c, h, r, q}

Step 6: Dequeue b, enqueue its unvisited neighbor (a)

Queue: [c, h, r, q, a]

Visited: {S, d, e, p, b, c, h, r, q, a}

Step 7: Dequeue c → a already visited (skip)

Queue: [h, r, q, a]

Step 8: Dequeue h → p, q already visited (skip)

Queue: [r, q, a]

Step 9: Dequeue r, enqueue f

Queue: [q, a, f]

Visited: {S, d, e, p, b, c, h, r, q, a, f}

Step 10: Dequeue q → no neighbors

Queue: [a, f]

Step 11: Dequeue a → no neighbors

Queue: [f]

Step 12: Dequeue f, enqueue G

Queue: [G]

Visited: {S, d, e, p, b, c, h, r, q, a, f, G}

Step 13: Dequeue G → no neighbors

Queue: []

☒ **BFS COMPLETE**

☒ **Final BFS Traversal Order**

$S \rightarrow d \rightarrow e \rightarrow p \rightarrow b \rightarrow c \rightarrow h \rightarrow r \rightarrow q \rightarrow a \rightarrow f \rightarrow G$

b) Questions Related to Breadth-First Search (BFS)

1 How much space does the fringe require?

- BFS stores **all nodes at the current depth level** in the fringe (queue).
- In the worst case, the number of nodes grows **exponentially** with depth.

$\text{Space Complexity of BFS} = O(b^d)$

Where:

- b = branching factor
- d = depth of the shallowest goal node

2 Is BFS complete?

Yes, BFS is complete

Reason:

- BFS explores nodes **level by level**
- If a solution exists at a finite depth, BFS is **guaranteed to find it**

(Assuming finite branching factor)

3 Is BFS optimal?

Yes, BFS is optimal under specific conditions

Explanation:

- BFS always finds the **shallowest solution**
- BFS is **optimal when all step costs are equal (unit cost)**

✗ If step costs are different → BFS is **not optimal**

✓ In that case → **Uniform Cost Search** should be used

Questions Related to Depth-First Search (DFS)

1 How much space does the fringe require?

- DFS stores only the nodes along the **current path** and their siblings.
- Memory usage is **much smaller than BFS**.

Space Complexity of DFS = $O(b \cdot d)$

Where:

- b = branching factor
- d = maximum depth of the search tree

2 Is DFS complete?

No, DFS is not complete (in general)

Reason:

- DFS may go **infinitely deep** along one branch
- It may **never reach the goal**, even if the goal exists elsewhere

✓ DFS is complete **only if**:

- The search space is **finite**, or
- A **depth limit** is imposed

3 Is DFS optimal?

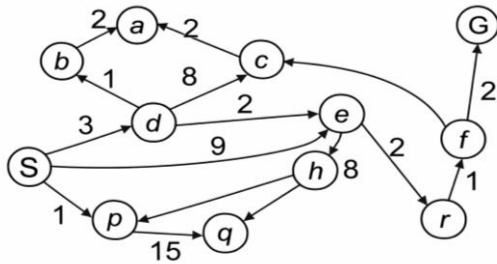
No, DFS is not optimal

Reason:

- DFS does **not guarantee the shortest path**
- It may find a **deeper, more expensive solution** first

Question 4

- a) What is the **difference between uninformed search and informed search**?
- b) Apply **Uniform Cost Search (UCS)** to the given graph and show all steps clearly.



a) Difference between Uninformed Search and Informed Search

Aspect	Uninformed Search	Informed Search
Definition	Search strategy that uses no additional knowledge about the problem other than the state space and goal test	Search strategy that uses heuristic (domain-specific) knowledge to guide the search
Guidance	Explores nodes without knowing how close they are to the goal	Uses a heuristic function to estimate closeness to the goal
Efficiency	Generally less efficient ; may explore many unnecessary nodes	More efficient ; reduces search space
Time & Space Cost	Usually higher time and memory consumption	Usually lower time and memory usage
Optimality	May or may not find optimal solution (depends on algorithm)	Can find optimal solution if heuristic is admissible
Completeness	Some algorithms are complete (e.g., BFS, UCS)	Some algorithms are complete (e.g., A*)
Examples	BFS, DFS, Uniform Cost Search	Greedy Best-First Search, A* Search

b) Apply Uniform Cost Search (UCS) to the given graph and show all steps clearly

Uniform Cost Search (UCS) – FULL PROCESS

1 Problem Definition

- **Start node:** S
- **Goal node:** G
- **Search strategy:** Uniform Cost Search (UCS)
- **Evaluation:** Expand the node with **lowest cumulative path cost**

2 Final Correct Graph (Directed, with Costs)

- $S \rightarrow p$ (1)
- $S \rightarrow d$ (3)
- $S \rightarrow e$ (9)
- $d \rightarrow b$ (1)
- $d \rightarrow c$ (8)
- $d \rightarrow e$ (2)
- $b \rightarrow a$ (2)
- $c \rightarrow a$ (2)
- $e \rightarrow h$ (8)
- $e \rightarrow r$ (2)
- $h \rightarrow p$ (4)
- $h \rightarrow q$ (4)
- $p \rightarrow q$ (15)
- $r \rightarrow f$ (1)
- $f \rightarrow c$ (3)
- $f \rightarrow G$ (2)

3 UCS Rules (write this in exam)

- UCS uses a **priority queue**
- Nodes are expanded in **increasing path cost**
- If a cheaper path to a node is found, update it
- Stop when **GOAL is dequeued**
- UCS guarantees the **optimal solution**

4 Step-by-Step UCS Process

Notation:

(node, total_cost_from_S)

◆ Step 1: Initialize

Priority Queue: [(S, 0)]

Expanded: {}

◆ Step 2: Expand S

Add: p (1)

Add: d (3)

Add: e (9)

Queue: [(p,1), (d,3), (e,9)]

Expanded: {S}

◆ Step 3: Expand p (cost = 1)

Add: q (1 + 15 = 16)

Queue: [(d,3), (e,9), (q,16)]

Expanded: {S, p}

◆ Step 4: Expand d (cost = 3)

Add: b (3 + 1 = 4)

Add: c (3 + 8 = 11)

Add: e (3 + 2 = 5) ← better than e(9)

Queue: [(b,4), (e,5), (c,11), (q,16)]

Expanded: {S, p, d}

◆ Step 5: Expand b (cost = 4)

Add: a (4 + 2 = 6)

Queue: [(e,5), (a,6), (c,11), (q,16)]

Expanded: {S, p, d, b}

◆ **Step 6: Expand e (cost = 5)**

Add: h ($5 + 8 = 13$)

Add: r ($5 + 2 = 7$)

Queue: [(a,6), (r,7), (c,11), (h,13), (q,16)]

Expanded: {S, p, d, b, e}

◆ **Step 7: Expand a (cost = 6)**

a has no outgoing edges

Queue: [(r,7), (c,11), (h,13), (q,16)]

Expanded: {S, p, d, b, e, a}

◆ **Step 8: Expand r (cost = 7)**

Add: f ($7 + 1 = 8$)

Queue: [(f,8), (c,11), (h,13), (q,16)]

Expanded: {S, p, d, b, e, a, r}

◆ **Step 9: Expand f (cost = 8)**

Add: G ($8 + 2 = 10$)

Add: c ($8 + 3 = 11$) → equal to existing c, ignore

Queue: [(G,10), (c,11), (h,13), (q,16)]

Expanded: {S, p, d, b, e, a, r, f}

◆ **Step 10: Expand G (GOAL FOUND)**

STOP SEARCH

5 Optimal Path Found by UCS

Tracing parents backward:

$$S \rightarrow d \rightarrow e \rightarrow r \rightarrow f \rightarrow G$$

6 Total Optimal Cost

$$3 + 2 + 2 + 1 + 2 = 10$$

7 UCS Node Expansion Order

$$S \rightarrow p \rightarrow d \rightarrow b \rightarrow e \rightarrow a \rightarrow r \rightarrow f \rightarrow G$$

8 Final Exam-Ready Conclusion

Uniform Cost Search expands nodes in order of increasing cumulative path cost and guarantees an optimal solution.

For the given graph, UCS finds the least-cost path

$S \rightarrow d \rightarrow e \rightarrow r \rightarrow f \rightarrow G$

with a total cost of 10.

Section B: Machine Learning

Question 5

a) What are the **main differences between supervised, unsupervised, and reinforcement learning**? Provide **one example for each type**.

b) A mechanical engineer sent you the following email:

"You may remember that yesterday we met at the World Scientific Conference. As I am working on devising new composite materials, I would like to use the K-Nearest Neighbors (K-NN) algorithm to build predictive models for classifying whether a material is thermally stable or not based on some material features. However, I am not familiar with how K-NN works. Can you explain the working process of the K-NN algorithm and give me advice on how to choose the value of K?"

Prepare a **detailed and professional reply** to this email.

c) What is **overfitting in machine learning**? How can it be **prevented**?

a) What are the main differences between supervised, unsupervised, and reinforcement learning? Provide one example for each type.

Machine learning algorithms are commonly categorized into **supervised learning**, **unsupervised learning**, and **reinforcement learning**, based on how they learn from data.

1. Supervised Learning: Supervised learning is a type of machine learning in which a model is trained using labeled data, where each input is paired with a known correct output. The model learns a mapping between inputs and outputs by analyzing example data and adjusting its parameters to minimize prediction error. Once trained, the model can predict outputs for new, unseen data. Supervised learning is commonly used for classification and regression tasks.

Examples:

- Email spam detection
- Image classification
- House price prediction
- Disease diagnosis

2. Unsupervised Learning: Unsupervised learning is a type of machine learning that works with unlabeled data, where no predefined output labels are provided. The algorithm analyzes the data to discover hidden patterns, structures, or relationships on its own without external guidance. It is mainly used for tasks such as clustering, association, and dimensionality reduction.

Examples:

- Clustering customers into groups based on buying behavior
- Grouping news articles by topic automatically
- Market basket analysis (finding items often bought together)
- Image compression using dimensionality reduction

3. Reinforcement Learning: Reinforcement learning is a type of machine learning in which an agent interacts with an environment by taking actions and learning from feedback in the form of rewards or penalties. The agent's goal is to learn an optimal policy that maximizes cumulative reward over time through trial and error, without being explicitly told the correct action.

Examples:

- **Game playing AI** (Chess, Go, Atari games)
- **Robot navigation** (learning to avoid obstacles)
- **Self-driving cars** (learning driving actions through feedback)
- **Recommendation systems** (improving suggestions based on user feedback)
- **Traffic signal control** (adjusting lights to reduce congestion)

b) Prepare a detailed and professional reply explaining the K-Nearest Neighbors (K-NN) algorithm and how to choose K

Below is a **formal professional email**, suitable for academic and industrial communication.

Email

Subject

Explanation of K-Nearest Neighbors (K-NN) Algorithm and Selection of K
Explanation of K-Nearest Neighbors (K-NN) Algorithm and Selection of K

Dear Sir,

It was a pleasure meeting you yesterday at the World Scientific Conference. Thank you for reaching out regarding the use of the K-Nearest Neighbors (K-NN) algorithm for classifying the thermal stability of composite materials.

K-Nearest Neighbors (K-NN) is a simple yet powerful supervised machine learning algorithm used primarily for classification and regression tasks. The algorithm does not explicitly build a model during training; instead, it stores all the training data and makes predictions based on similarity.

The working process of the K-NN algorithm can be summarized as follows:

1. First, all training samples with known class labels (for example, “thermally stable” or “not thermally stable”) are stored.
2. When a new material sample is provided, the algorithm calculates the distance between this sample and all stored training samples. Common distance measures include Euclidean distance.
3. The algorithm then selects the K nearest data points (neighbors) based on the smallest distances.
4. For classification tasks, the class that appears most frequently among these K neighbors is assigned to the new sample.

Regarding the choice of the value of K, it plays a critical role in the performance of the K-NN algorithm:

- A small value of K (such as $K = 1$) makes the model highly sensitive to noise and may lead to overfitting.
- A large value of K smooths the decision boundary but may cause underfitting by ignoring important local patterns.
- In practice, K is often chosen using cross-validation, testing multiple values of K and selecting the one that yields the best validation accuracy.
- A common rule of thumb is to start with $K \approx \sqrt{N}$, where N is the number of training samples.

Additionally, it is important to normalize or standardize your material features before applying K-NN, since the algorithm is distance-based and sensitive to feature scales.

I hope this explanation helps you in applying K-NN effectively to your materials classification problem. Please feel free to contact me if you need further clarification or assistance.

Kind regards,
Adnan Zaman Niloy

c) What is overfitting in machine learning? How can it be prevented?

Overfitting

Definition:

Overfitting occurs when a machine learning model learns the training data too well, including noise and irrelevant details, instead of capturing the underlying patterns. As a result, the model performs very well on the training data but shows poor performance on unseen or test data, leading to weak generalization

Symptoms:

- Very high training accuracy
- Low testing or validation accuracy

Causes of Overfitting

- Too complex model
- Too many features
- Insufficient training data
- Noise in the dataset

Methods to Prevent Overfitting

1. **Use More Training Data**
 - Reduces model dependency on noise.
2. **Feature Selection**
 - Remove irrelevant features.
3. **Regularization**
 - Penalizes large model parameters (e.g., L1, L2 regularization).
4. **Cross-Validation**
 - Helps select models that generalize well.
5. **Early Stopping**
 - Stops training before the model overfits.
6. **Simpler Models**
 - Prefer simpler models when possible.

Question 6

- a) What is **Machine Learning (ML)**? Why is the **trend of machine learning growing so rapidly**? What are its **real-life applications**?
 - b) Describe **different types of machine learning algorithms** with suitable examples.
 - c) How is **machine learning different from general (traditional) programming**?
 - d) Why can **linear regression not be used for a classification task**?
-

a) What is Machine Learning (ML)? Why is the trend of machine learning growing so rapidly? What are its real-life applications?

What is Machine Learning (ML)?

Machine Learning (ML) is a subfield of Artificial Intelligence that enables computer systems to learn from data and improve their performance automatically without being explicitly programmed for every task. In machine learning, algorithms analyze historical data, identify patterns, and make predictions or decisions when new data is provided.

Why is the trend of Machine Learning growing so rapidly?

The rapid growth of machine learning is due to several key factors:

1. **Availability of Big Data**
 - Massive amounts of data are generated daily from social media, sensors, mobile devices, and online systems.
2. **Increased Computational Power**
 - Powerful GPUs, cloud computing, and distributed systems enable fast model training.
3. **Advancements in Algorithms**
 - Improved algorithms such as deep learning provide higher accuracy.
4. **Industry Demand**
 - Businesses use ML for automation, prediction, and decision-making.
5. **Open-Source Tools and Frameworks**
 - Libraries like TensorFlow, PyTorch, and Scikit-learn make ML accessible.

Real-Life Applications of Machine Learning

Machine learning is widely used in real-world applications, including:

- **Healthcare:** Disease diagnosis, medical image analysis
- **Finance:** Fraud detection, credit scoring
- **Transportation:** Self-driving cars, traffic prediction
- **E-commerce:** Recommendation systems (Amazon, Netflix)

- **Education:** Personalized learning platforms
- **Security:** Face recognition, intrusion detection
- **Agriculture:** Crop yield prediction, disease detection

b) Describe different types of machine learning algorithms with suitable examples.

Machine learning algorithms are mainly classified into the following types:

1. Supervised Learning

Definition:

Supervised learning uses **labeled data**, where the correct output is known during training.

Tasks:

- Classification
- Regression

Examples:

- Linear Regression (predicting house prices)
- Logistic Regression (spam detection)
- Decision Trees (loan approval)

2. Unsupervised Learning

Definition:

Unsupervised learning works with **unlabeled data** and discovers hidden patterns.

Tasks:

- Clustering
- Dimensionality reduction

Examples:

- K-Means clustering (customer segmentation)
- Hierarchical clustering
- Principal Component Analysis (PCA)

3. Reinforcement Learning

Definition:

Reinforcement learning involves an **agent** that learns by interacting with an environment and receiving **rewards or penalties**.

Tasks:

- Sequential decision-making
- Control problems

Examples:

- Game-playing AI
- Robot navigation
- Traffic signal control

4. Semi-Supervised Learning

Definition:

Uses a **small amount of labeled data** and a large amount of unlabeled data.

Example:

- Speech recognition systems

c) How is machine learning different from general (traditional) programming?

In **traditional programming**, a human programmer writes explicit rules and instructions based on domain knowledge. These rules are applied to input data to produce the desired output. If the output is incorrect or the requirements change, the programmer must manually modify the rules and logic.

In **machine learning**, the system is not explicitly programmed with rules. Instead, the programmer provides input data along with the correct outputs (in supervised learning). The machine learning algorithm automatically learns patterns and relationships from the data and builds a model. This model is then used to make predictions on new, unseen data. The system improves its performance as more data becomes available.

Traditional programming is suitable for **simple, well-defined problems**, whereas machine learning is effective for **complex, data-driven problems** such as image recognition, speech processing, and recommendation systems.

Comparison Table

Traditional Programming	Machine Learning
Programmer writes explicit rules and logic	Model learns rules automatically from data
Input data and program produce output	Input data and expected output are used to train a model
Works well for clearly defined problems	Works well for complex and data-driven problems
Rules must be manually updated	Model improves automatically with more data
Less flexible	More adaptable and scalable

d) Why can linear regression not be used for a classification task?

Linear regression is **not suitable for classification** because:

1. **Outputs are not bounded**
Linear regression can produce values from $-\infty$ to $+\infty$, while classification requires outputs within a fixed range (e.g., 0 or 1).
2. **No probabilistic interpretation**
Classification often needs class probabilities, but linear regression does not naturally provide valid probability estimates.
3. **Poor decision boundaries**
Linear regression minimizes squared error, which is not appropriate for separating classes and can lead to incorrect or unstable decision boundaries.
4. **Sensitive to outliers**
Squared error loss is highly affected by outliers, which can significantly distort classification results.
5. **Better alternatives exist**
Algorithms like **Logistic Regression**, **SVM**, and **Decision Trees** are specifically designed for classification and handle class boundaries more effectively.

Correct Alternative

- **Logistic Regression** is used instead because it:
 - Outputs values between 0 and 1
 - Models probability
 - Is designed for classification tasks

Question 7

a) Suppose medical researchers hired you to understand the relationship between **drug dosage and patients' blood pressure**. What **type of machine learning problem** is this?

b) You are given the following dataset for the above problem. Consider a **linear regression model**. What will be the **training loss (MSE)** when **W = 0, 1, 2, 3, and 4**?

Dosage (mg)	Blood Pressure (Diastolic)
5	90
13	92
19	93
10	91

c) What is the **learning rate in machine learning**? Discuss the **problems associated with learning rate selection** and explain how we can choose a **better learning rate**.

a) Suppose medical researchers hired you to understand the relationship between drug dosage and patients' blood pressure. What type of machine learning problem is this?

This is a **supervised learning – regression problem**.

Explanation

- The **input** (drug dosage) is a **numerical/continuous variable**.
- The **output** (blood pressure) is also a **continuous numerical value**.
- The goal is to **predict a continuous outcome** based on given input data.

Since labeled input–output pairs are available and the output is continuous, this problem is best modeled using **regression**, specifically **linear regression**.

b) Consider a linear regression model. What will be the training loss (MSE) when $W = 0, 1, 2, 3$, and 4?

Given Dataset

Dosage (x)	Blood Pressure (y)
5	90
13	92
19	93
10	91

Model Assumption

We use a simple linear regression model **without bias**:

$$\hat{y} = W \cdot x$$

Loss Function (Mean Squared Error)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where $n = 4$.

Case 1: $W = 0$

Predictions:

$$\hat{y} = 0$$

Squared errors:

- $(90 - 0)^2 = 8100$
- $(92 - 0)^2 = 8464$
- $(93 - 0)^2 = 8649$

- $(91 - 0)^2 = 8281$

$$\text{MSE} = \frac{8100 + 8464 + 8649 + 8281}{4} = \frac{33494}{4} = \boxed{8373.5}$$

Case 2: $W = 1$

Predictions:

$$\hat{y} = x$$

Squared errors:

- $(90 - 5)^2 = 7225$
- $(92 - 13)^2 = 6241$
- $(93 - 19)^2 = 5476$
- $(91 - 10)^2 = 6561$

$$\text{MSE} = \frac{25493}{4} = \boxed{6373.25}$$

Case 3: $W = 2$

Predictions:

$$\hat{y} = 2x$$

Squared errors:

- $(90 - 10)^2 = 6400$
- $(92 - 26)^2 = 4356$
- $(93 - 38)^2 = 3025$
- $(91 - 20)^2 = 5041$

$$\text{MSE} = \frac{18822}{4} = \boxed{4705.5}$$

Case 4: $W = 3$

Predictions:

$$\hat{y} = 3x$$

Squared errors:

- $(90 - 15)^2 = 5625$
- $(92 - 39)^2 = 2809$

- $(93 - 57)^2 = 1296$
- $(91 - 30)^2 = 3721$

$$\text{MSE} = \frac{13451}{4} = \boxed{3362.75}$$

Case 5: $W = 4$

Predictions:

$$\hat{y} = 4x$$

Squared errors:

- $(90 - 20)^2 = 4900$
- $(92 - 52)^2 = 1600$
- $(93 - 76)^2 = 289$
- $(91 - 40)^2 = 2601$

$$\text{MSE} = \frac{9390}{4} = \boxed{2347.5}$$

Summary Table

W MSE

0 8373.5

1 6375.75

2 4705.5

3 3362.75

4 **2347.5 (Lowest)**

→ **Conclusion:** As W increases, the training loss decreases, and among the given values, $W = 4$ gives the lowest MSE.

c) What is the learning rate in machine learning? Discuss the problems associated with learning rate selection and explain how we can choose a better learning rate.

Learning Rate

The **learning rate (η)** is a hyperparameter that controls **how much the model's parameters are updated** during each step of optimization (e.g., gradient descent).

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial W}$$

Problems with Learning Rate Selection

1. Learning Rate Too Small

- Training becomes very slow
- Takes many iterations to converge
- May get stuck in local minima

2. Learning Rate Too Large

- Model may overshoot the minimum
- Loss may oscillate or diverge
- Training becomes unstable

How to Choose a Better Learning Rate

1. Learning Rate Tuning

- Try multiple values (e.g., 0.1, 0.01, 0.001)

2. Learning Rate Schedulers

- Reduce learning rate gradually during training

3. Adaptive Optimizers

- Algorithms like Adam, RMSProp automatically adjust learning rates

4. Learning Rate Finder

- Start small and increase gradually while observing loss behavior.

Question 8

a) Suppose we have a dataset where **each data sample has two features**.

1. Design a **linear model** for this problem and draw the **computational graph**.
2. Assume the two learnable parameters are **W_1 and W_2** , and the loss function is **Mean Squared Error (MSE)**.

Find

$$\frac{\partial \text{Loss}}{\partial W_1} \text{ and } \frac{\partial \text{Loss}}{\partial W_2}$$

when

- $X_1 = 2$

- $X_2 = 3$
- $Y = 7$
- Initial values: $W_1 = 3, W_2 = 2$

b) What **types of problems** can be effectively solved using **unsupervised learning methods**?

a) Linear model design, computational graph, and gradient calculation

Step 1: Design a linear model (two features)

Since each data sample has **two features**, a suitable **linear regression model** (without bias) is:

$$\hat{Y} = W_1 X_1 + W_2 X_2$$

where

- X_1, X_2 = input features
- W_1, W_2 = learnable parameters
- $\hat{Y}$ = predicted output

Step 2: Loss function (Mean Squared Error)

For a **single training sample**, Mean Squared Error (MSE) is:

$$\text{Loss} = (Y - \hat{Y})^2$$

Step 3: Computational graph (conceptual)

The computation flows as follows:

- Inputs X_1, X_2
- Multiply by weights W_1, W_2
- Add results → prediction $\hat{Y}$
- Compare with true value Y
- Compute squared error (Loss)

Step 4: Given values

- $X_1 = 2$
- $X_2 = 3$
- $Y = 7$
- $W_1 = 3$

- $W_2 = 2$

Step 5: Forward pass (prediction)

$$\hat{Y} = (3 \times 2) + (2 \times 3) = 6 + 6 = 12$$

Step 6: Compute loss

$$\text{Loss} = (Y - \hat{Y})^2 = (7 - 12)^2 = (-5)^2 = 25$$

Step 7: Gradient calculation

Using the chain rule:

$$\frac{\partial \text{Loss}}{\partial W_i} = 2(\hat{Y} - Y) \frac{\partial \hat{Y}}{\partial W_i}$$

Since

$$\hat{Y} = W_1 X_1 + W_2 X_2$$

we have:

$$\frac{\partial \hat{Y}}{\partial W_1} = X_1 \text{ and } \frac{\partial \hat{Y}}{\partial W_2} = X_2$$

Gradient with respect to W_1

$$\frac{\partial \text{Loss}}{\partial W_1} = 2(12 - 7)(2) = 2(5)(2) = \boxed{20}$$

Gradient with respect to W_2

$$\frac{\partial \text{Loss}}{\partial W_2} = 2(12 - 7)(3) = 2(5)(3) = \boxed{30}$$

Final Answer (Part a)

$$\boxed{\frac{\partial \text{Loss}}{\partial W_1} = 20 \text{ and } \frac{\partial \text{Loss}}{\partial W_2} = 30}$$

b) What types of problems can be effectively solved using unsupervised learning methods?

Unsupervised learning is used when no labeled output data is available. It is effective for discovering hidden structures and patterns in data.

1 Clustering Problems

Grouping data points into clusters such that points within the same cluster are **more similar** to each other than to those in other clusters.

What it discovers:

- Natural groupings in data
- Similarity-based structure

Examples:

- Customer segmentation in marketing
- Document or news article clustering
- Image segmentation
- Grouping genes with similar expression patterns

2 Pattern Discovery (Association Analysis)

Finding **hidden relationships and frequent patterns** among variables in large datasets.

What it discovers:

- Co-occurrence patterns
- Dependency relationships

Examples:

- Market basket analysis (items frequently bought together)
- Recommendation systems
- Web usage mining

3 Dimensionality Reduction

Purpose:

Reducing the number of features while **preserving important information** in the data.

What it helps with:

- Data visualization
- Noise reduction
- Faster computation
- Avoiding overfitting

Examples:

- Visualizing high-dimensional data in 2D/3D

- Feature extraction in image processing
- Reducing redundant features

Anomaly / Outlier Detection

Identifying data points that **deviate significantly** from normal patterns.

What it discovers:

- Rare or abnormal events
- Errors or unexpected behavior

Examples:

- Fraud detection in banking
- Network intrusion detection
- Fault detection in industrial systems
- Medical anomaly detection

5 Data Compression

Representing data in a **compact form** while retaining essential information.

What it achieves:

- Reduced storage requirements
- Efficient transmission of data

Examples:

- Image and video compression
- Signal compression
- Feature encoding in deep learning

6 Feature Learning

Automatically learning **useful representations** from raw data without labels.

Examples:

- Learning image features
- Word embeddings in NLP

Unsupervised learning is especially useful for exploratory data analysis, where the goal is to understand data structure, detect patterns, and discover meaningful groupings without predefined labels.

Class Test–1

Question 1

- a) Define Artificial Intelligence (AI) and explain its main types with suitable examples.
 - b) What tasks can Artificial Intelligence perform in today's world?
 - c) When performing an action, how does a reflex agent behave compared to a planning agent? Explain with an example.
-

Question 2

- a) What is a **search problem** in Artificial Intelligence? What are the **main components** of a search problem?
- b) Explain the difference between a **state space graph** and a **search tree** in the context of a search problem.
- c) Consider a **Tic-Tac-Toe game environment** with the following parameters:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing directions: North, South, East, West (N, S, E, W)

Now calculate the following:

- Total number of **world states**
 - Number of **states for path-finding**
 - Number of **states for the eat-all-dots problem**
-

A **perceptron** is the **simplest artificial neural network model** used for **binary classification**. It takes input features, multiplies them by weights, adds a bias, and passes the result through an **activation function** to produce an output.

Class Test-02

Question 1

- a) Define an Artificial Neural Network (ANN). Explain the structural difference between a single-layer perceptron and a Multi-Layer Perceptron (MLP).
 - b) Explain why a single-layer perceptron cannot solve the XOR problem, whereas a Multi-Layer Perceptron (MLP) can.
 - c) Draw the perceptron model (showing inputs, weights, and threshold values) to represent the following Boolean logic gates:
 - AND gate
 - OR gate
-

a)

Artificial Neural Network (ANN) : An **Artificial Neural Network (ANN)** is a computational model inspired by the structure and functionality of **human brain**, made up of interconnected **artificial neurons**. It processes information by passing inputs through weighted connections and activation functions to produce outputs, and it learns by adjusting these weights.

Structural Difference: Single-Layer Perceptron vs Multi-Layer Perceptron (MLP)

1 Single-Layer Perceptron

Structure:

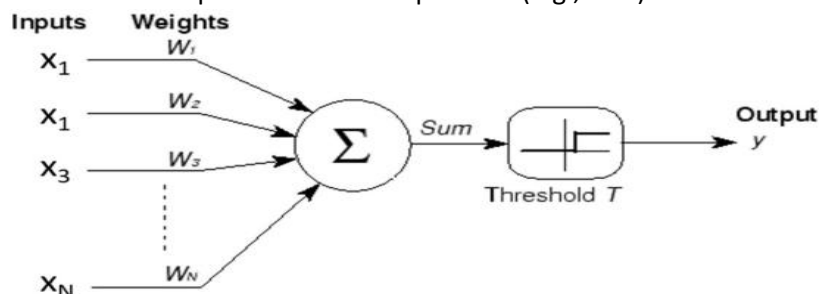
- One **input layer**
- One **output layer**
- **No hidden layer**

Characteristics:

- Each input is directly connected to the output neuron
- Uses a simple activation function (usually step function)
- Can solve **only linearly separable problems**

Limitation:

- Cannot learn complex or non-linear patterns (e.g., XOR)



2 Multi-Layer Perceptron (MLP)

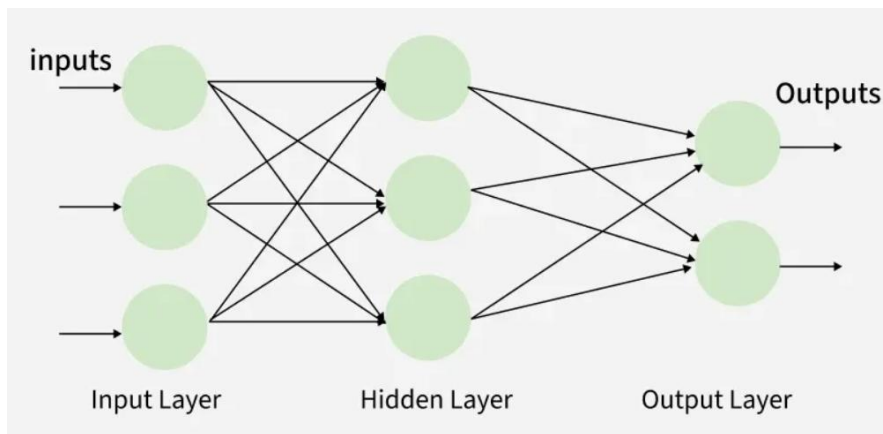
Structure:

- One **input layer**
- One or more **hidden layers**
- One **output layer**

Characteristics:

- Hidden layers allow feature extraction
- Uses non-linear activation functions (ReLU, sigmoid, tanh)
- Trained using **backpropagation**

- Can solve **both linear and non-linear problems**



b) Why a Single-Layer Perceptron Cannot Solve XOR but an MLP Can

The XOR (exclusive OR) function outputs **1 only when the inputs are different**. Truth Table for XOR ($X \oplus Y$)

X	Y	$X \vee Y$	$\bar{X} \vee \bar{Y}$	$X \oplus Y$
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

A **single-layer perceptron** fails to solve the XOR problem because **XOR is not linearly separable**.

1 Only One Straight Line

- A single-layer perceptron adds inputs using weights and applies a step function.
- This creates **only one straight decision line**.
- It can separate data **only if a straight line is enough**.

2 XOR Cannot Be Separated by a Straight Line

- In XOR, the output **1s are on opposite corners**.
- The output **0s are also on opposite corners**.
- No single straight line can correctly separate these two groups.

3 What Happens in Learning

- There is **no set of weights and bias** that can classify all XOR inputs correctly.
- Because of this, the perceptron **fails to learn XOR**.
- The training process **does not converge**.

Why a Multi-Layer Perceptron (MLP) Can Solve XOR

An **MLP overcomes the limitations** of a single-layer perceptron by introducing additional layers and non-linearity.

1 Hidden Layers

- An MLP contains **one or more hidden layers**.
- Hidden neurons learn **intermediate representations** of the input.

2 Multiple Linear Boundaries

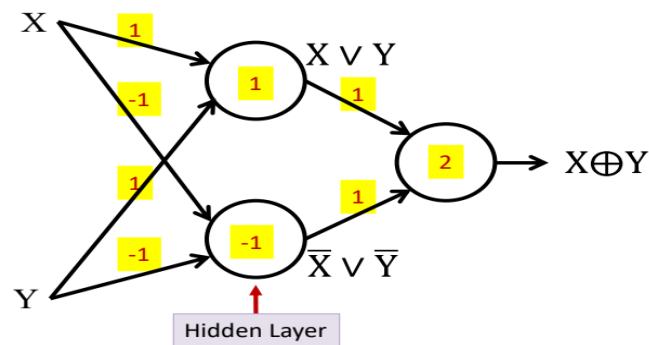
- Each hidden neuron creates its own linear boundary.
- When combined, these boundaries form a **non-linear decision surface**.

3 Non-Linear Activation Functions

- Activation functions such as **ReLU, sigmoid, or tanh** introduce non-linearity.
- This allows the network to model **complex patterns** like XOR.

4 XOR Interpretation Using Hidden Neurons

- One hidden neuron learns $x_1 \vee x_2$
- Another hidden neuron learns $x_1 \wedge x_2$
- The output neuron combines these to produce **XOR**



c) Perceptron Model for Boolean Logic Gates

A **perceptron** computes:

$$y = \begin{cases} 1, & \text{if } w_1x_1 + w_2x_2 \geq \theta \\ 0, & \text{otherwise} \end{cases}$$

Where:

- x_1, x_2 = inputs
- w_1, w_2 = weights

- θ = threshold

1 AND Gate

Truth Table (AND)

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

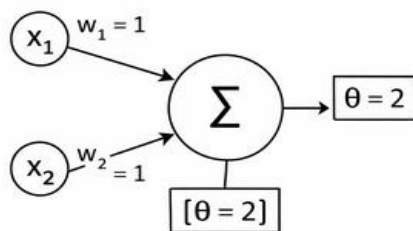
Perceptron Parameters

- $w_1 = 1$
- $w_2 = 1$
- $\theta = 2$

Decision Rule

$$y = 1 \text{ if } x_1 + x_2 \geq 2$$

Perceptron Diagram (AND)



✦ Output is **1** only when both inputs are 1.

2 OR Gate

Truth Table (OR)

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Perceptron Parameters

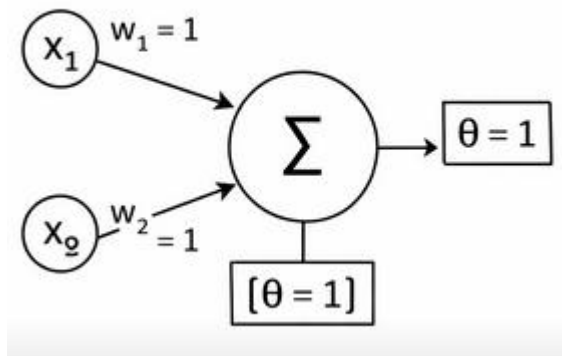
- $w_1 = 1$

- $w_2 = 1$
- $\theta = 1$

Decision Rule

$$y = 1 \text{ if } x_1 + x_2 \geq 1$$

Perceptron Diagram (OR)



✦ Output is **1** when at least one input is 1.

Question 2

The following dataset is given:

ID	Study Time	Attended Class	Pass
D1	High	Yes	Yes
D2	High	Yes	Yes
D3	Low	Yes	No
D4	Low	No	No
D5	High	No	Yes
D6	Low	No	No

- Calculate the entropy of the entire dataset before any splitting.
- Calculate the entropy of the child nodes obtained by splitting the dataset based on the attributes "Study Time" and "Attended Class".

c) Compute the Information Gain for both attributes. Based on your calculations, determine which attribute should be selected as the root node of the decision tree.

(a) Entropy of the Entire Dataset

Step 1: Count class labels

- Yes = 3 (D1, D2, D5)
- No = 3 (D3, D4, D6)
- Total = 6

Step 2: Formula

$$Entropy(S) = -p^+ \log_2 p^+ - p^- \log_2 p^-$$

Step 3: Probabilities

$$p^+ = \frac{3}{6} = 0.5, p^- = \frac{3}{6} = 0.5$$

Step 4: Calculation

$$\begin{aligned} Entropy(S) &= -0.5 \log_2(0.5) - 0.5 \log_2(0.5) \\ Entropy(S) &= -0.5(-1) - 0.5(-1) = 1.0 \end{aligned}$$

$$\boxed{Entropy(S) = 1.0}$$

(b) Entropy After Splitting

◆ Split on Study Time

Study Time = High

Pass

Yes

Yes

Yes

- Yes = 3, No = 0

$$Entropy = 0$$

Study Time = Low

Pass

No

No

No

- Yes = 0, No = 3

$$Entropy = 0$$

◆ Split on Attended Class

Attended Class = Yes

Pass

Yes

Yes

No

- Yes = 2, No = 1

$$Entropy = -\frac{2}{3}\log_2 \frac{2}{3} - \frac{1}{3}\log_2 \frac{1}{3}$$
$$Entropy = 0.918$$

Attended Class = No

Pass

No

Yes

No

- Yes = 1, No = 2

$$Entropy = -\frac{1}{3}\log_2 \frac{1}{3} - \frac{2}{3}\log_2 \frac{2}{3}$$
$$Entropy = 0.918$$

(c) Information Gain Calculation

Formula

$$Gain(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

◆ Information Gain for Study Time

Weighted entropy

$$\frac{3}{6}(0) + \frac{3}{6}(0) = 0$$

Gain

$$Gain(S, StudyTime) = 1.0 - 0 = \boxed{1.0}$$

◆ Information Gain for Attended Class

Weighted entropy

$$\frac{3}{6}(0.918) + \frac{3}{6}(0.918) = 0.918$$

Gain

$$Gain(S, AttendedClass) = 1.0 - 0.918 = \boxed{0.082}$$

Final Comparison

Attribute	Information Gain
-----------	------------------

Study Time	1.000
------------	--------------

Attended Class	0.082
----------------	-------